

实验二：原生应用开发 —— 实验报告

实验项目基本信息

项目	内容
实验编号	d20301035102、d20301009202
实验名称	原生应用开发
姓名	汪明昊
学号	2023210624
技术栈	微信小程序
开发工具	微信开发者工具
学时分配	4学时
实验类型	验证型
对应课程目标	课程目标1

一、需求分析阶段

1.1 功能需求

开发"智慧校园"登录模块，学生在登录页面输入学号和密码，系统验证成功后跳转到个人中心页面，显示个性化的欢迎信息和用户资料。

序号	功能	描述
1	学号输入	文本输入框，支持学号输入
2	密码输入	密码输入框，支持密码隐藏/显示切换
3	登录按钮	触发登录验证逻辑

序号	功能	描述
4	输入验证	不能为空、长度不少于4位
5	登录成功跳转	跳转至个人中心页面
6	数据传递	将学号传递到个人中心页面
7	退出登录	个人中心支持退出返回登录页

1.2 界面需求

- Material Design 风格设计（渐变背景 + 圆角卡片）
- 输入框带标签和占位提示文字
- 密码隐藏/显示切换按钮
- 登录按钮 loading 状态
- 个人中心展示欢迎信息、个人信息卡片、功能菜单

1.3 跳转需求

- 登录成功 → 个人中心（URL 传递学号参数）
- 个人中心 → 退出登录 → 返回登录页

1.4 输入验证规则

校验项	规则	错误提示
学号为空	<code>username.trim() === ''</code>	"学号不能为空"
学号长度不足	<code>username.trim().length < 4</code>	"学号长度不能少于4位"
密码为空	<code>password === ''</code>	"密码不能为空"
密码长度不足	<code>password.length < 4</code>	"密码长度不能少于4位"

二、AI辅助编码阶段

2.1 项目创建

使用微信开发者工具创建小程序项目，项目结构如下：

```
miniprogram/
├─ app.js           # 小程序入口，全局生命周期
├─ app.json         # 全局配置（页面路由、导航栏样式）
├─ app.wxss         # 全局样式（CSS 变量定义）
├─ project.config.json # 微信开发者工具项目配置
├─ sitemap.json     # 站点地图
├─ pages/
│   ├─ login/       # 登录页面
│   │   ├─ login.js  # 登录逻辑
│   │   ├─ login.json # 页面配置
│   │   ├─ login.wxml # 登录界面模板
│   │   └─ login.wxss # 登录页面样式
│   └─ home/        # 个人中心页面
│       ├─ home.js   # 个人中心逻辑
│       ├─ home.json  # 页面配置
│       ├─ home.wxml  # 个人中心界面模板
│       └─ home.wxss  # 个人中心样式
```

2.2 登录页面实现

界面布局（login.wxml）

使用 WXML 模板语法构建登录界面，包含学号输入框、密码输入框（带显示/隐藏切换）、登录按钮。通过 `wx:if` 条件渲染显示验证错误信息。

```

<view class="login-container">
  <view class="login-card">
    <view class="login-header">
      <text class="title">智慧校园</text>
      <text class="subtitle">欢迎登录</text>
    </view>

    <view class="form-group">
      <view class="input-wrapper">
        <text class="input-label">学号</text>
        <input class="input-field" type="text" placeholder="请输入学号"
          value="{{username}}" bindinput="onUsernameInput"
          bindblur="onUsernameBlur" maxlength="{{20}}" />
        <text class="error-text" wx:if="{{usernameError}}">{{usernameError}}</text>
      </view>

      <view class="input-wrapper">
        <text class="input-label">密码</text>
        <input class="input-field" type="{{showPassword ? 'text' : 'password'}}"
          placeholder="请输入密码" value="{{password}}"
          bindinput="onPasswordInput" bindblur="onPasswordBlur"
          maxlength="{{20}}" />
        <view class="password-toggle" bindtap="togglePassword">
          <text>{{showPassword ? '🔒' : '👁'}}</text>
        </view>
        <text class="error-text" wx:if="{{passwordError}}">{{passwordError}}</text>
      </view>
    </view>

    <button class="login-btn" type="primary" bindtap="onLogin"
      loading="{{isLoading}}" disabled="{{isLoading}}">
      登 录
    </button>
  </view>
</view>

```

登录逻辑 (login.js)

核心逻辑包含实时输入监听、失焦校验、登录动作处理和数据传递：

```
Page({
  data: {
    username: '',
    password: '',
    usernameError: '',
    passwordError: '',
    showPassword: false,
    isLoading: false
  },

  // 学号输入
  onUsernameInput(e) {
    this.setData({ username: e.detail.value, usernameError: '' })
  },

  // 密码输入
  onPasswordInput(e) {
    this.setData({ password: e.detail.value, passwordError: '' })
  },

  // 学号失焦校验
  onUsernameBlur() {
    const { username } = this.data
    if (!username.trim()) {
      this.setData({ usernameError: '学号不能为空' })
    } else if (username.trim().length < 4) {
      this.setData({ usernameError: '学号长度不能少于4位' })
    }
  },

  // 密码失焦校验
  onPasswordBlur() {
    const { password } = this.data
    if (!password) {
      this.setData({ passwordError: '密码不能为空' })
    } else if (password.length < 4) {
      this.setData({ passwordError: '密码长度不能少于4位' })
    }
  },

  // 密码显示切换
  togglePassword() {
    this.setData({ showPassword: !this.data.showPassword })
  }
})
```

```

},

// 登录验证
validateInput() {
  const { username, password } = this.data
  let isValid = true

  if (!username.trim()) {
    this.setData({ usernameError: '学号不能为空' })
    isValid = false
  } else if (username.trim().length < 4) {
    this.setData({ usernameError: '学号长度不能少于4位' })
    isValid = false
  }

  if (!password) {
    this.setData({ passwordError: '密码不能为空' })
    isValid = false
  } else if (password.length < 4) {
    this.setData({ passwordError: '密码长度不能少于4位' })
    isValid = false
  }

  return isValid
},

// 登录动作
onLogin() {
  if (!this.validateInput()) return

  this.setData({ isLoading: true })
  wx.showLoading({ title: '登录中...', mask: true })

  setTimeout(() => {
    wx.hideLoading()
    const { username, password } = this.data

    if (username.trim() === 'admin' && password === '123456') {
      wx.showToast({ title: '登录成功', icon: 'success', duration: 1000 })
      setTimeout(() => {
        wx.navigateTo({
          url: `/pages/home/home?username=${encodeURIComponent(username.trim())}`
        })
      })
    }
  })
}

```

```
    }, 1000)
  } else {
    wx.showToast({ title: '用户名或密码错误', icon: 'error', duration: 2000 })
  }
  this.setData({ isLoading: false })
}, 1500)
}
})
```

2.3 个人中心页面实现

界面布局（home.wxml）

```
<view class="home-container">
  <view class="home-header">
    <view class="avatar">  </view>
    <text class="welcome-text">欢迎 {{username}}</text>
    <text class="role-text">在校学生</text>
  </view>

  <view class="info-card">
    <view class="info-title">个人信息</view>
    <view class="info-item">
      <text class="info-label">学号</text>
      <text class="info-value">{{username}}</text>
    </view>
    <view class="info-item">
      <text class="info-label">登录时间</text>
      <text class="info-value">{{loginTime}}</text>
    </view>
    <view class="info-item">
      <text class="info-label">状态</text>
      <text class="info-value status-online">在线</text>
    </view>
  </view>

  <view class="menu-list">
    <view class="menu-item">  个人资料</view>
    <view class="menu-item">  我的课表</view>
    <view class="menu-item">  成绩查询</view>
    <view class="menu-item">  系统设置</view>
  </view>

  <button class="logout-btn" bindtap="onLogout">退出登录</button>
</view>
```


业务逻辑 (home.js)

```
Page({
  data: {
    username: '',
    loginTime: ''
  },

  onLoad(options) {
    const username = decodeURIComponent(options.username || '未知用户')
    const now = new Date()
    const loginTime = `${now.getFullYear()}-${String(now.getMonth() + 1)
      .padStart(2, '0')}-${String(now.getDate())
      .padStart(2, '0')} ${String(now.getHours())
      .padStart(2, '0')}:${String(now.getMinutes())
      .padStart(2, '0')}:${String(now.getSeconds())
      .padStart(2, '0')}`

    this.setData({ username, loginTime })
  },

  onLogout() {
    wx.showModal({
      title: '提示',
      content: '确定要退出登录吗?',
      success: (res) => {
        if (res.confirm) {
          wx.navigateBack()
        }
      }
    })
  }
})
```

关键技术点

- **页面跳转**: 使用 `wx.navigateTo()` 从登录页跳转到个人中心
- **数据传递**: 通过 URL query string `?username=xxx` 传递学号, 接收端用 `onLoad(options)` 获取
- **返回上一页**: 使用 `wx.navigateBack()` 实现退出登录返回

三、测试验证阶段

3.1 测试用例设计

编号	测试场景	输入条件	预期结果
TC-01	学号为空	学号留空，点击登录	显示红色错误提示「学号不能为空」
TC-02	密码为空	输入学号，密码留空，点击登录	显示红色错误提示「密码不能为空」
TC-03	学号长度不足	学号输入少于4位（如 123 ）	显示红色错误提示「学号长度不能少于4位」
TC-04	密码长度不足	密码输入少于4位（如 12 ）	显示红色错误提示「密码长度不能少于4位」
TC-05	错误凭证	学号 admin ，密码 wrong	Toast 提示「用户名或密码错误」
TC-06	正确凭证	学号 admin ，密码 123456	Toast 「登录成功」→ 跳转个人中心
TC-07	退出登录	点击退出登录按钮	弹窗确认 → 返回登录页面

3.2 测试结果截图

TC-01：学号为空

输入学号留空后点击登录，输入框边框变红，下方显示红色提示文字「学号不能为空」。

11:59



智慧校园登录



智慧校园

欢迎登录

学号

请输入学号

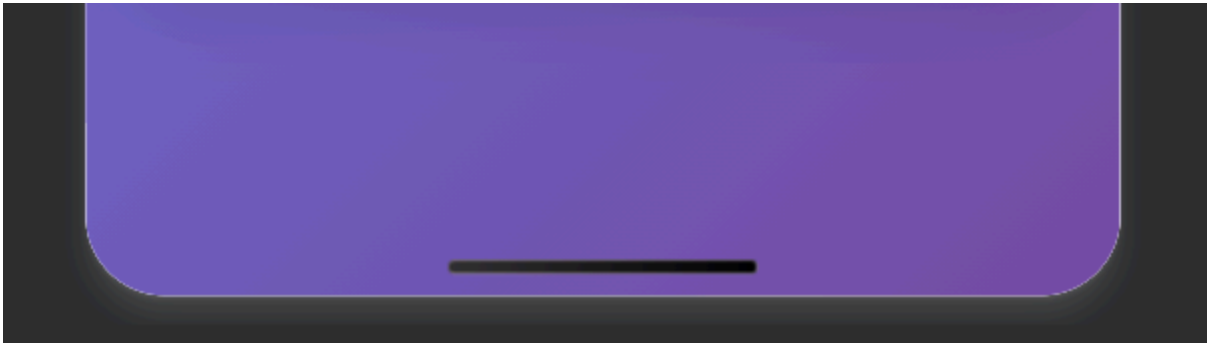
学号不能为空

密码

请输入密码



登录



TC-02：密码为空

输入学号但密码留空后点击登录，密码输入框边框变红，下方显示红色提示文字「密码不能为空」。

12:00



智慧校园登录



智慧校园

欢迎登录

学号

admin

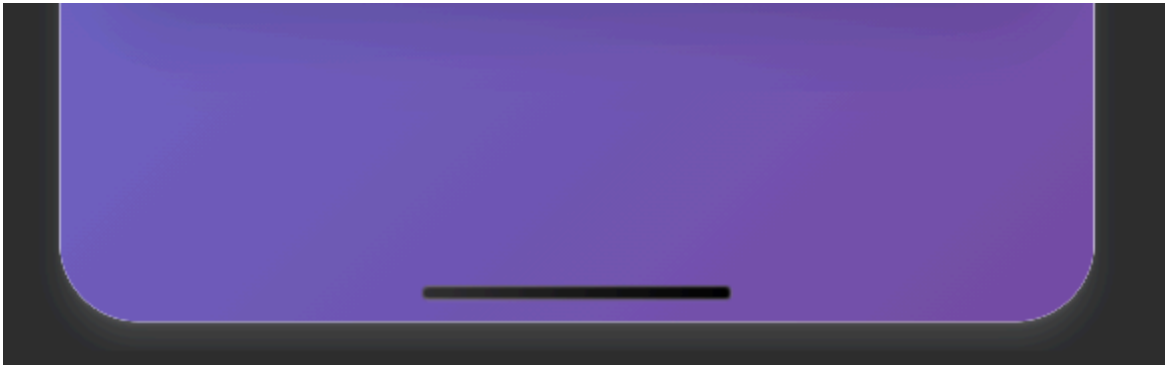
密码

请输入密码



密码不能为空

登录



TC-03：学号长度不足

输入学号 123 （不足4位）后点击登录，显示红色错误提示「学号长度不能少于4位」。

12:00



智慧校园登录



智慧校园

欢迎登录

学号

adm

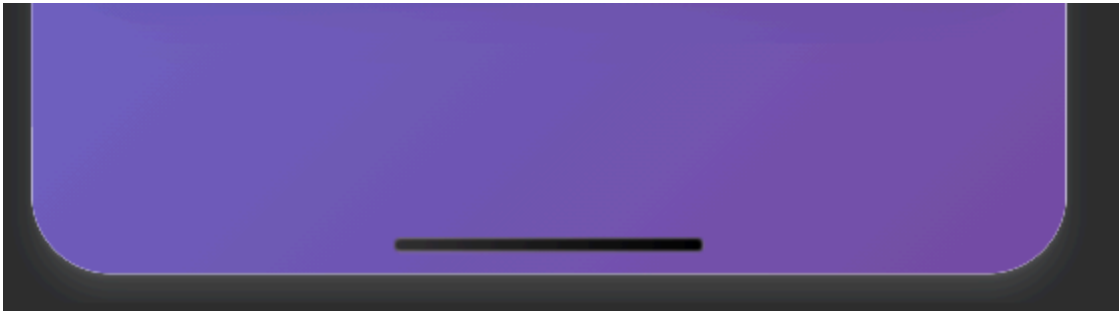
学号长度不能少于4位

密码

123456



登录



TC-04：密码长度不足

输入密码 12 （不足4位）后点击登录，显示红色错误提示「密码长度不能少于4位」。

12:01



智慧校园登录



智慧校园

欢迎登录

学号

admin

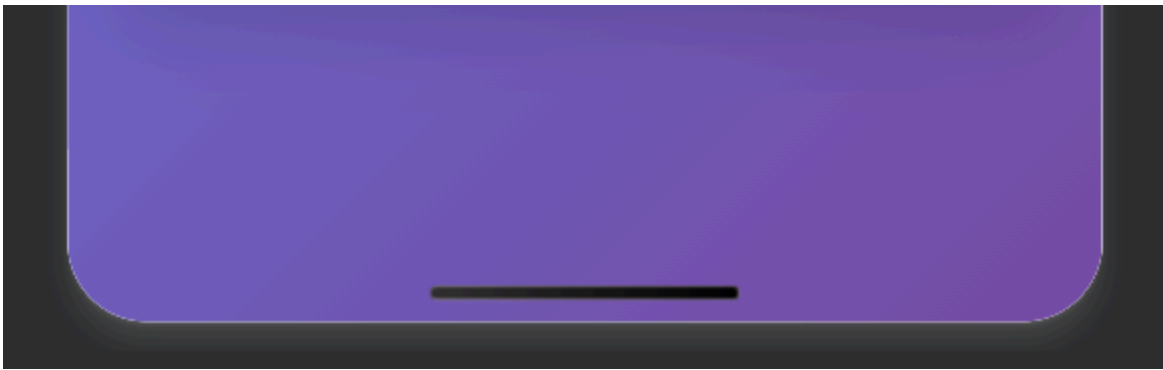
密码

123|



密码长度不能少于4位

登录



TC-05：错误凭证

输入学号 `admin`、密码 `wrong` 后点击登录，系统弹出 Toast 提示「用户名或密码错误」。

12:02



智慧校园登录



学号

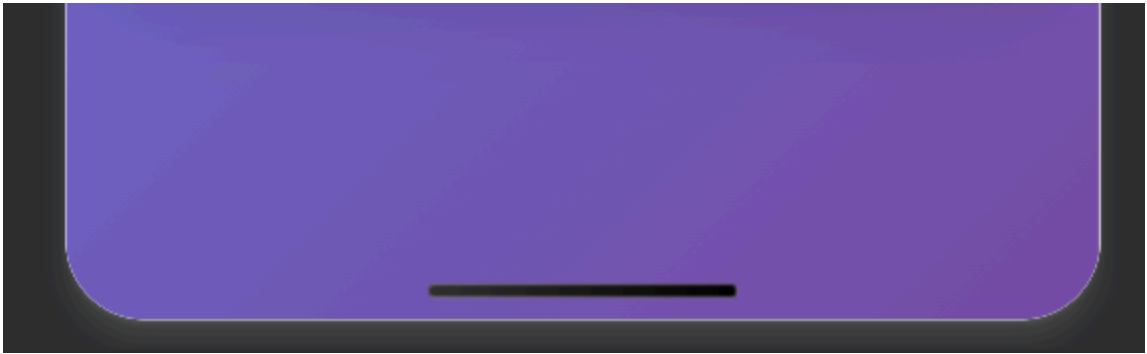
admin

密码

wrong



登录



TC-06：正确凭证登录成功

输入学号 admin、密码 123456 后点击登录，系统弹出绿色 Toast「登录成功」→ 跳转至个人中心页面，显示欢迎信息、学号、登录时间和功能菜单。

11:58



个人中心



欢迎 admin

在校学生

个人信息

学号

admin

登录时间

2026-06-01 11:59:23

状态

在线

 个人资料

 我的课表

 成绩查询

 系统设置

退出登录

3.3 测试结果汇总

编号	测试场景	实际结果	状态
TC-01	学号为空	正确显示「学号不能为空」	✓ 通过
TC-02	密码为空	正确显示「密码不能为空」	✓ 通过
TC-03	学号长度不足	正确显示「学号长度不能少于4位」	✓ 通过
TC-04	密码长度不足	正确显示「密码长度不能少于4位」	✓ 通过
TC-05	错误凭证	正确 Toast 提示错误	✓ 通过
TC-06	正确凭证	正确跳转个人中心，数据传递正常	✓ 通过
TC-07	退出登录	弹窗确认，正确返回登录页	✓ 通过

测试通过率：7/7（100%）

四、平台差异对比

4.1 微信小程序 vs Android 原生 vs iOS 原生

对比项	Android (Kotlin)	iOS (SwiftUI)	微信小程序
开发语言	Kotlin	Swift	JavaScript
UI 框架	XML Layout / Jetpack Compose	SwiftUI	WXML + WXSS
页面跳转	Intent + startActivity()	NavigationLink	wx.navigateTo() / wx.navigateBack()
数据传递	Intent.putExtra() / Bundle	@Binding / @State	URL query string + options 参数
输入验证	EditText.setError()	@State 绑定 + 条件判断	setData() + wx:if 条件渲染

对比项	Android (Kotlin)	iOS (SwiftUI)	微信小程序
Toast 提示	Toast.makeText()	.alert() 修饰符	wx.showToast()
对话框	AlertDialog.Builder	.alert()	wx.showModal()
运行环境	Android 虚拟机 (JVM)	iOS 模拟器	微信内置浏览器引擎 (双端统一)
跨平台能力	仅 Android	仅 iOS	一套代码，Android + iOS 双端运行
安装包体积	APK ≥ 4MB	IPA ≥ 2MB	无需安装，即用即走
审核发布	Google Play / 应用商店	App Store	微信审核，无需商店上架
调试工具	Android Studio	Xcode	微信开发者工具

4.2 技术选型分析

微信小程序优势：

- 一套代码即可覆盖 Android 和 iOS 两端
- 无需安装，用户通过微信扫码即用
- 开发门槛低，前端开发者上手快
- 微信生态内分享传播便捷

原生开发优势：

- 性能更优，可访问底层硬件 API
- 不受微信平台限制，用户体验更独立
- 可上架应用商店获取自然流量

五、部署运维阶段

5.1 调试版本

在微信开发者工具中点击「预览」，扫码即可在手机上运行调试版本。也可通过「上传」将代码上传至微信后台，提交审核后发布正式版本。

5.2 项目配置

```
{
  "compileType": "miniprogram",
  "libVersion": "3.3.4",
  "projectname": "SmartCampusLogin",
  "setting": {
    "es6": true,
    "postcss": true,
    "minified": true,
    "enhance": true
  }
}
```

六、总结与思考

6.1 实验总结

通过本次实验，我掌握了以下技能：

1. **微信小程序页面开发**：熟悉了 WXML 模板语法、WXSS 样式编写、JavaScript 逻辑层的 Page 生命周期
2. **输入验证逻辑**：实现了空值校验和长度校验，结合 `bindinput` 和 `bindblur` 事件实现实时反馈
3. **页面跳转与数据传递**：使用 `wx.navigateTo()` 配合 URL query string 实现跨页面数据传递，使用 `onLoad(options)` 接收参数
4. **用户交互体验**：实现了 loading 状态、Toast 提示、Modal 确认对话框等交互组件
5. **平台差异理解**：对比了微信小程序与 Android/iOS 原生开发在语言、框架、运行环境等方面的差异

6.2 课后思考

1. 原生开发相比跨平台的优势和劣势

- **优势**：原生开发性能最优，能充分利用平台 SDK 和硬件能力，用户体验最流畅；不受第三方容器限制
- **劣势**：需要维护 Android 和 iOS 两套代码，开发成本高；学习曲线陡峭，需要掌握两种语言和框架

2. 如何选择原生开发 vs 跨平台开发

- 对性能要求极高、需要深度硬件访问的场景（如游戏、AR）优先选原生
- 快速验证 MVP、企业级管理应用、小程序生态内的场景适合微信小程序
- 追求体验同时希望代码复用的项目可考虑 Flutter 或 React Native

3. AI 辅助代码生成对开发效率的影响

- AI 能快速生成页面骨架代码，大幅减少模板编写时间
- 开发者可将精力集中在业务逻辑和用户体验优化上
- 但需要开发者具备审查和调试能力，AI 生成的代码并非 100% 正确

附录：完整代码文件清单

文件	说明
miniprogram/app.js	小程序入口文件
miniprogram/app.json	全局配置文件
miniprogram/app.wxss	全局样式文件
miniprogram/pages/login/login.wxml	登录页面模板
miniprogram/pages/login/login.js	登录页面逻辑
miniprogram/pages/login/login.json	登录页面配置
miniprogram/pages/login/login.wxss	登录页面样式
miniprogram/pages/home/home.wxml	个人中心页面模板
miniprogram/pages/home/home.js	个人中心页面逻辑
miniprogram/pages/home/home.json	个人中心页面配置
miniprogram/pages/home/home.wxss	个人中心页面样式
miniprogram/project.config.json	项目工具配置